

CS3334 - Data Structures

Lab 2

Outline

- Program Complexity Exercises
- Largest Subsequence
- Manipulate Stacks

Basic understanding about Analysis

- Why do we care about Big-Oh Analysis?
- Why can we write $n^2+3n+1=O(n^2)$? We know that $n^2+3n+1>n^2$!
- We said that $n^4=o(2^n)$, but if I set $n=3$, I have $3^4>2^3$, how does it come?

Basic understanding about Analysis

- Suppose program 1 has worst case running time $T1(n)=3n^2+100n\log n+9$, program 2 has worst case running time $T2(n)=2^n+n$ and they can output the same results. Which program do you choose? Explain the reasons for the choice.
- Suppose program 1 has best case running time $T1(n)=3n^2$, program 2 has best case running time $T2(n)=n$ and they can output the same results. Which program do you choose? Explain the reasons for your answer.

Exercise 1

Provide the Big-O analysis for each of the expressions below

	Expression
1	$5 + 0.001n^3 + 0.025n$
2	$500n + 100n^{1.5} + 50n \log_{10} n$
3	$0.3n + 5n^{1.5} + 2.5 \cdot n^{1.75}$
4	$n^2 \log_2 n + n(\log_2 n)^2$
5	$n \log_3 n + n \log_2 n$
6	$3 \log_8 n + \log_2 \log_2 \log_2 n$
7	$100n + 0.01n^2$
8	$0.01n + 100n^2$
9	$2n + n^{0.5} + 0.5n^{1.25}$
10	$0.01n \log_2 n + n(\log_2 n)^2$
11	$100n \log_3 n + n^3 + 100n$
12	$0.003 \log_4 n + \log_2 \log_2 n$

Exercise 2

Analyze the running time of the following program

(a)

```
Void function1(int n)
{
    int i,j;
    int x=0;
    for(i=0;i<n;i++)
    {
        If(x<100)
            x++;
        for(j=0;j<n;j++)
            x+=j;
    }
}
```

(b)

```
Void function2 (int n)
{
    int i,j;
    int x=200;
    for(i=0;i<n;i++)
    {
        If(x>100)
            x--;
        else
            for(j=0;j<n;j++)
                x+=j;
    }
}
```

Exercise 2

Analyze the running time of the following program

(c)

```
void function3(int n) {  
    if (n==1)  
        return;  
    for (int i=1; i<=n; i++) {  
        for (int j=1; j<=n; j++) {  
            print("*");  
            break;  
        }  
    }  
}
```

(d)

```
void function4(int n, int choice) {  
    int x = 0;  
    if (choice==1) {  
        for (int i=1; i<=n; i++)  
            for (int j=1; j<=20; j++)  
                for (int k=n; k>1; k=k/2)  
                    x+=5;  
    }  
    else {  
        for (int j=1; j<=n*n; j++)  
            for (int k=0; k<n; k++)  
                x+=3;  
    }  
}
```

Exercise 3

Knowing the recurrence equations below, provide the big-O analysis.

$$T(1) = 1$$

$$T(n) = 2 T(n/2) + n$$

$$T(n) = O(?)$$

Exercise 4

Given an array *arr*[], find the Previous Smaller Element (PSE) for every element in the array.

The Previous Smaller Element of an element *x* is defined as the first element to its left in the array that is smaller than *x*.

If no such element exists for a particular position, the PSE should be considered as -1.

Make your implementation to have **O(n)** time complexity (n is the array size).

Examples:

Input: *arr*[] = [1, 6, 2]

Output: [-1, 1, 1]

```
vector<int> prevSmaller(vector<int>& arr) {  
    ... // put your codes here  
    ... // return the final result array  
}
```

Input: *arr*[] = [1, 5, 0, 3, 4, 5]

Output: [-1, 1, -1, 0, 3, 4]

Largest Subsequence

Description

A subsequence of a string **x** can be made by erasing some (possibly all or none) of the letters in **x**.

For example, "opt" is a subsequence of "computer", while "rt" is not.

Now, we want to find the lexicographical largest subsequence from a given string.

For example, the sorted subsequences of "test" are:

"" (empty string), "e", "es", "est", "et", "s", "st", "t", "te", "tes", "test", "tet", "ts", "tst" and "tt".

And "tt" is the largest subsequence here, so print it out.

Largest Subsequence

Input

On the first line of input, there is an integer N , representing the number of test cases.

The next N line, there is a string x . The length of x is between 1 and 52. Only lowercase characters appear in x .

Output

For each test case, output the largest subsequence on each line.

Sample Input

```
4
test
a
bun
yukkurishiteittene
```

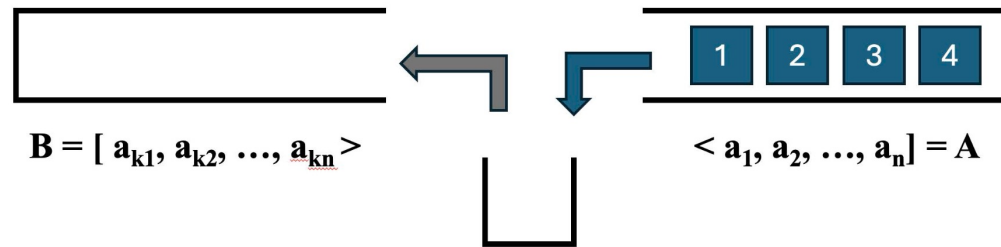
Output for Sample Input

```
tt
a
un
yuutttne
```

Manipulate Stacks

Description

After learning stack, the widely used data structure, Icy plan to play with it. The rule is as follows: There are 3 stacks, **A**, **B** and **S**, where stack **B** and **S** are empty initially.



There are only two kinds of movement.

- (1) Top element of **A** can only be moved onto the top of **S**.
- (2) Top element of **S** can only be moved onto the top of **B**.

By repeating (1) and (2) until **A** and **S** are empty, all elements in **A** will be moved to **B** and the elements in **B** are permuted (the order may not be the same). So here comes the problem. Given the initial order of elements in stack **A** and a final order of elements in **B**, can you cleverly judge whether the final order is possible to achieve?

Manipulate Stacks

Input

The input contains multiple test cases.

The first line of input is an integer T ($1 \leq T \leq 10$) representing the number of test cases.

For each test case, the first line gives an integer n ($1 \leq n \leq 3000$) indicating the number of elements in stack **A**, the following line gives n integers representing the corresponding elements in stack **A** (first element is bottom, last element is top and we guarantee that all the elements are distinct).

Then, the next line contains an integer m ($m \leq 200$) telling you how many permutations you have to judge and in each of the following m lines there are n integers indicating the desired elements to be tested in stack **B**.

Output

If the permutation is possible to achieve, print "Aye" otherwise print "Impossible" in a separate line.

Manipulate Stacks

Sample Input	Sample Output
1 5 1 2 3 4 5 3 1 2 3 4 5 1 5 4 2 3 3 2 1 4 5	Aye Impossible Aye

Hints

For the last permutation "3 2 1 4 5", it can be achieved by the following operations:

A→**S**:5

A→**S**:4

A→**S**:3

S→**B**:3

A→**S**:2

S→**B**:2

A→**S**:1

S→**B**:1

S→**B**:4

S→**B**:5